

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, faint, stylized circular shape in the center, with the density of the dots being higher in the center and fading out towards the edges.

HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.





ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Heuristic Algorithm

ONE LOVE. ONE FUTURE.

Heuristic algorithm

- Heuristic algorithms are strategies designed to efficiently tackle complex optimization problems by providing approximate solutions when exact methods are impractical.
- This is achieved by trading optimality, completeness, accuracy, or precision for speed.
- The objective of a heuristic is to produce a solution in a reasonable time frame that is good enough for solving the problem at hand. This solution may not be the best of all the solutions to this problem, or it may simply approximate the exact solution. But it is still valuable because finding it does not require a prohibitively long time.



Outline

- **Greedy Algorithm**
 - Introduction to greedy algorithm
 - Example 1: Fractional Knapsack
 - Example 2: 0/1 Knapsack
 - Example 3: Multi-Knapsack
 - Example 4: TSP
 - Example 5: Set cover problem
 - Exercises
- Local Search



Greedy algorithm

Greedy algorithms are a class of algorithms that make **locally optimal** choices at each step with the hope of finding a **global optimum** solution.

- **S**: a solution is represented by a set of components
- **C**: set of candidates of components to be added to the solution
- **select(C)**: select the most promising component among candidates
- **solution(S)**: return true if **S** is a solution to the original problem
- **feasible(S)**: return true if **S** does not violate any constraints

```
Greedy() {  
    S = {};  
    while C  $\neq$   $\emptyset$  and  
        not solution(S){  
        x = select(C);  
        C = C \ {x};  
        if feasible(S  $\cup$  {x}) {  
            S = S  $\cup$  {x};  
        }  
    }  
    return S;  
}
```



Outline

- **Greedy Algorithm**

- Introduction to greedy algorithm
- **Example 1: Fractional Knapsack**
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- Example 4: TSP
- Example 5: Set cover problem
- Exercises

- Local Search



Fractional knapsack

Problem: There are now a set of items where each item has its value and weight. You have a knapsack and hope to put in as many valuable items as possible. However, the knapsack's capacity is limited to some weight W that the total weight of selected items should not exceed W . The goal is to select a collection of items achieving the maximum total value while satisfying the capacity restriction. This problem in general is called the knapsack problem and it has many variations. In the fractional setting here, each item could be arbitrarily divided so that only a fraction of the item is taken. For example, suppose item A has weight 4 and value 10. Then a $1/2$ fraction of item A will occupy 2 weight and has value 5.

Example:

- **Input:** value = [60, 100, 120], weight = [10, 20, 30], weight capacity = 50
- **Output:** 240
- **Explanation:** By taking items of weight 10 and 20 kg and $2/3$ fraction of 30 kg. Total price will be $60+100+(2/3)(120) = 240$



Fractional knapsack

- Idea: Sort items in decreasing order based on the ratio of their value to their weight



Fractional knapsack

```
def fractionalKnapsack(val, wt, capacity):  
    n = len(val)  
  
    # Create list to store value and weight  
    # items[i][0] = value, items[i][1] = weight  
    items = [[val[i], wt[i]] for i in range(n)]  
  
    # Sort items based on value-to-weight ratio in descending order  
    items.sort(key=lambda x: x[0]/x[1], reverse=True)  
  
    res = 0.0  
    currentCapacity = capacity  
  
    # Process items in sorted order  
    for i in range(n):  
  
        # If we can take the entire item  
        if items[i][1] <= currentCapacity:  
            res += items[i][0]  
            currentCapacity -= items[i][1]
```

```
        # Otherwise take a fraction of the item  
        else:  
            res += (1.0 * items[i][0] / items[i][1]) * currentCapacity  
  
            # Knapsack is full  
            break  
  
    return res  
  
if __name__ == "__main__":  
    val = [60, 100, 120]  
    wt = [10, 20, 30]  
    capacity = 50  
  
    print(fractionalKnapsack(val, wt, capacity))
```



Outline

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- **Example 2: 0/1 Knapsack**
- Example 3: Multi-Knapsack
- Example 4: TSP
- Example 5: Set cover problem
- Exercises

- Local Search



0/1 Knapsack

- **Problem:** Given n items where each item has some weight and profit associated with it and given a bag with capacity W , [i.e., the bag can hold at most W weight in it]. The task is to put the items into the bag such that the sum of profits associated with them is the maximum possible.
- **Example:**
 - **Input:** $W = 4$, $\text{profit}[] = [1, 2, 3]$, $\text{weight}[] = [4, 5, 1]$
 - **Output:** 3
 - **Explanation:** There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So, the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.



0/1 Knapsack

- **Idea 1:** Sort items in decreasing order based on their value
- **Idea 2:** Sort items in increasing order based on their weight
- **Idea 3:** Sort items in decreasing order based on the ratio of their value to their weight
- **Idea 4:** The best one among the solutions returned from above ideas



0/1 Knapsack

```
def Knapsack(val, wt, capacity, idea):
    n = len(val)

    # Create list to store value and weight
    # items[i][0] = value, items[i][1] = weight
    items = [[val[i], wt[i]] for i in range(n)]
    # Sort items based on value-to-weight ratio in descending order
    if idea == 1:
        items.sort(key=lambda x: x[0], reverse=True)
    elif idea == 2:
        items.sort(key=lambda x: x[1], reverse=False)
    else:
        items.sort(key=lambda x: x[0]/x[1], reverse=True)

    res = 0.0
    currentCapacity = capacity
    # Process items in sorted order
    for i in range(n):
        # If we can take the entire item
        if items[i][1] <= currentCapacity:
            res += items[i][0]
            currentCapacity -= items[i][1]
        # Otherwise take a fraction of the item
        else:
            # Knapsack is full
            break
    return res
```

```
if __name__ == "__main__":
    val = [60, 100, 120]
    wt = [10, 20, 30]
    capacity = 50

    sol1 = Knapsack(val, wt, capacity, 1)
    sol2 = Knapsack(val, wt, capacity, 2)
    sol3 = Knapsack(val, wt, capacity, 3)
    sol4 = max(max(sol1, sol2), sol3)
    print(sol1, sol2, sol3, sol4)
```



Outline

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- **Example 3: Multi-Knapsack**
- Example 4: TSP
- Example 5: Set cover problem
- Exercises

- Local Search



Multi-Knapsack problem

- **Problem:** Given unlimited number of bins having capacity Q and n items $1, 2, \dots, n$ in which the weight of item i is $w(i)$. How to put these n items into bins such that the total weight of items put into each bin cannot exceed Q and the number of bins used is minimal.



Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    for i = 1 to n do{
        b = select(load,w(i));
        if b = NULL then {
            binNum = binNum + 1;
            b = binNum;
            load[b] = 0;
        }
        binOfItem[i] = b;
        load[b] = load[b] + w[i];
    }
    return binOfItem;
}
```

```
selectFirstFitBin(load,w){
    for b = 1 to binNum do {
        if load[b] + w <= Q then
            return b;
    }
    return NULL;
}
```

```
selectBestFitBin(load,w){
    R = -∞; sel_b = NULL
    for b = 1 to binNum do {
        if load[b] + w <= Q and (R < Q-load[b]-w) then
            R = Q - load[b] - w; sel_b = b;
    }
    return sel_b;
}
```



Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    sort items in an decreasing order of
    weights; (suppose  $w(1) \geq \dots \geq w(n)$ )
    for i = 1 to n do{
        b = select(load,w(i));
        if b = NULL then {
            binNum = binNum + 1;
            b = binNum;
            load[b] = 0;
        }
        binOfItem[i] = b;
        load[b] = load[b] + w[i];
    }
    return binOfItem;
}
```

```
selectFirstFitBin(load,w){
    for b = 1 to binNum do {
        if load[b] + w <= Q then
            return b;
    }
    return NULL;
}
```

```
selectBestFitBin(load,w){
    R =  $-\infty$ ; sel_b = NULL
    for b = 1 to binNum do {
        if load[b] + w <= Q and (R < Q-load[b]-w) then
            R = Q - load[b] - w; sel_b = b;
    }
    return sel_b;
}
```



Multi-Knapsack problem

```
greedyMNS(){
    binNum = 0
    candidates = {1,...,n}
    while (candidates not empty){
        R =  $\infty$ ; sel_bin = NULL; sel_item = 0;
        for i in candidates do{
            for b = 1 to binNum do{
                if load[b] + w(i)  $\leq$  Q and (Q - load[b] - w(i) < R then{
                    R = Q - load[b] - w(i); sel_bin = b; sel_item = i;
                }
            }
        }
        if (sel_bin = NULL){
            binNum = binNum + 1
        }else{
            candidates.remove(sel_item);
            binOfItem[sel_item] = sel_bin; load[sel_bin] = load[sel_bin] + w(sel_item);
        }
    }
    return binOfItem;
}
```



Outline

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- **Example 4: TSP**
- Example 5: Set cover problem
- Exercises

- Local Search



TSP

- **Problem:** Given n points $1, 2, \dots, n$ in which $d(i, j)$ is the distance from point i to point j . Find the shortest closed tour starting from 1 visiting other points and terminating at 1 such that the total travel distance is minimal.



TSP

- Idea 1:
 - The tour is initialized by point 1
 - At each step, select the nearest point to the last point of the tour under construction and add this point to the end of the tour
- Idea 2:
 - Apply idea 1 with different starting point
 - Select the shortest tour
- Idea 3:
 - Build a minimum spanning tree T for the graph G
 - Perform DFS traverse tree T so that a visit sequence of points
- Idea 4: Shortest one among three solutions provided by above ideas



Outline

- **Greedy Algorithm**

- Introduction to greedy algorithm
- Example 1: Fractional Knapsack
- Example 2: 0/1 Knapsack
- Example 3: Multi-Knapsack
- Example 4: TSP
- **Example 5: Set cover problem**
- Exercises

- Local Search



Set cover problem

- **Problem:** Suppose we are going to form a team to accomplish a mission. Team members are selected from a group of candidates each of whom specializes in several skills (such as reconnaissance, combat, medical support, etc.). The objective is to make the team as small as possible while satisfying that every required skill is owned by at least one team member. And this is one situation that needs an algorithm for set cover problem.

Set cover problem

- **Idea:** Select members with the largest set of needed skills.

